



# 数据结构

## Data Structure

许 蕾

[xlei@nju.edu.cn](mailto:xlei@nju.edu.cn)



# 第五章 树与二叉树



- 树和森林的概念
- 二叉树
- 二叉树遍历
- 线索化二叉树
- 树与森林
- 堆
- Huffman树

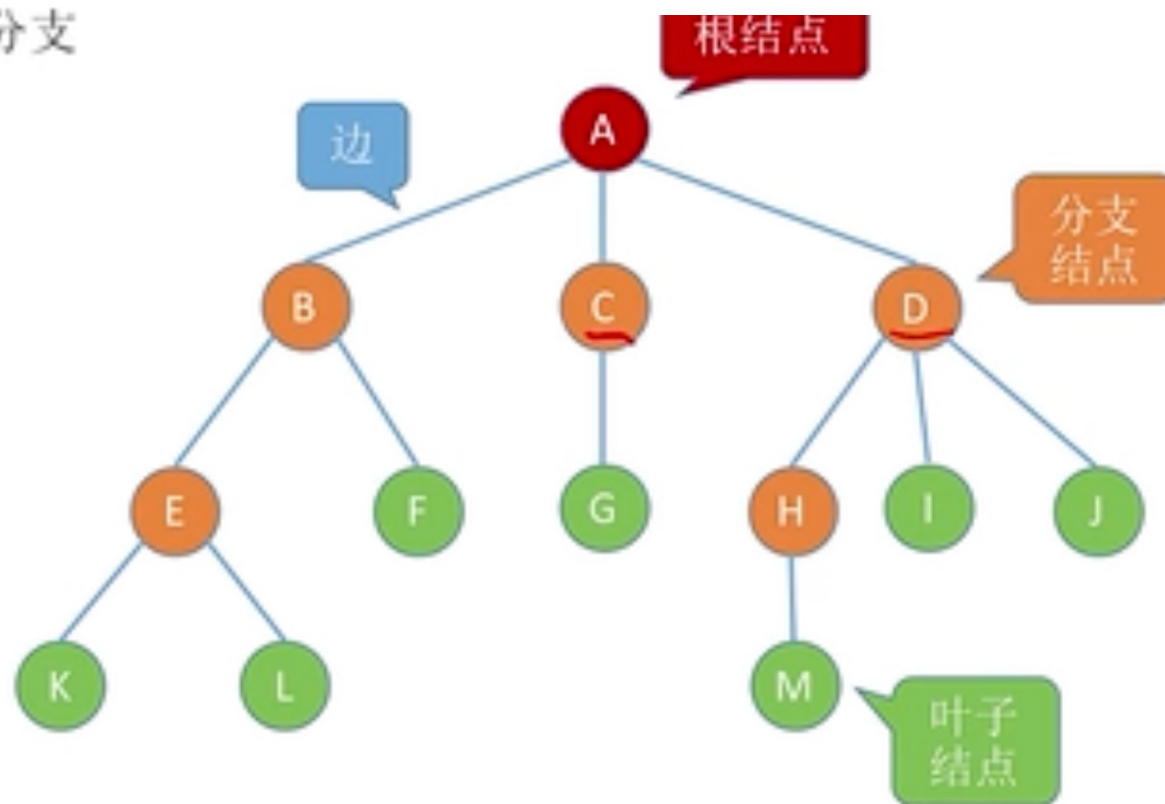


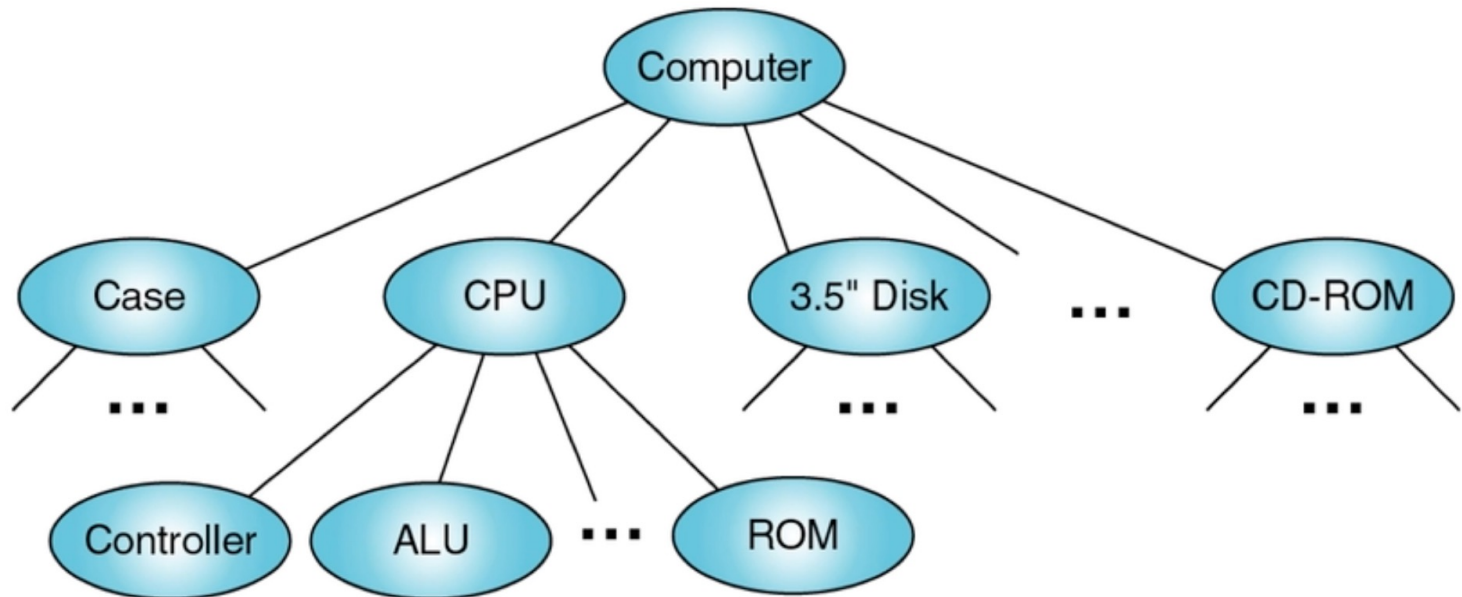
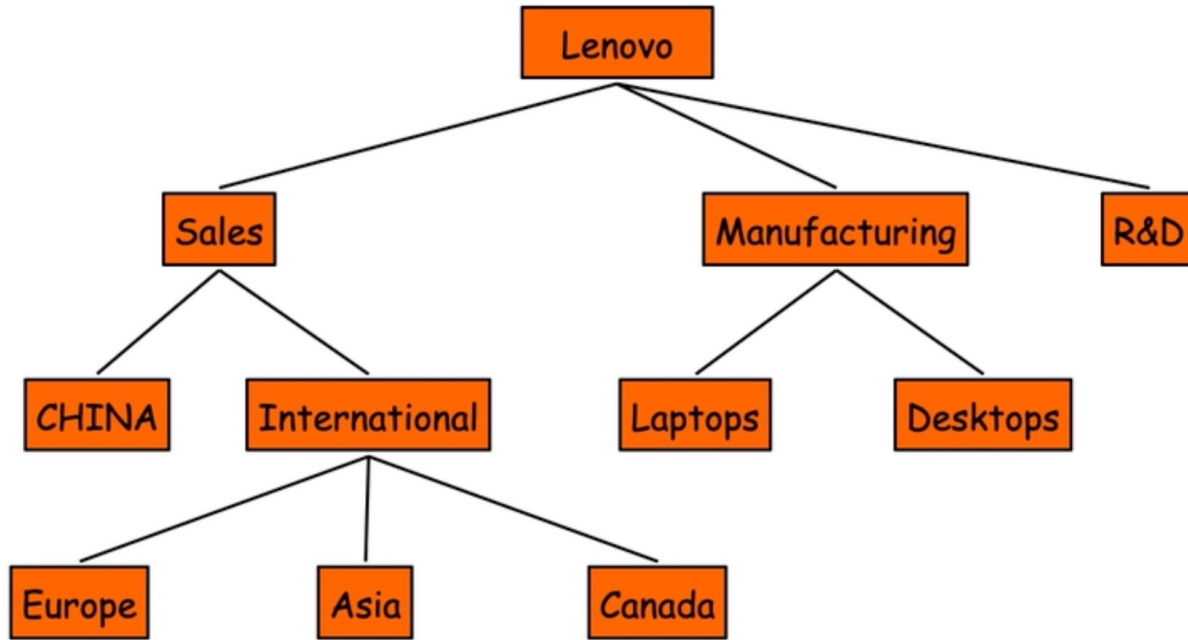
- 线性结构和表结构（一对一）不适合描述有分支结构的数据。
- 分支关系定义的层次结构（一对多）
  - 祖先-后代
  - 上级-下属
  - 整体-部分
- 树有着广泛应用
  - 文件系统、数据库系统
  - 编译系统
  - 算法设计与分析





树：从树根生长，逐级分支







## 5.1 树和森林的概念



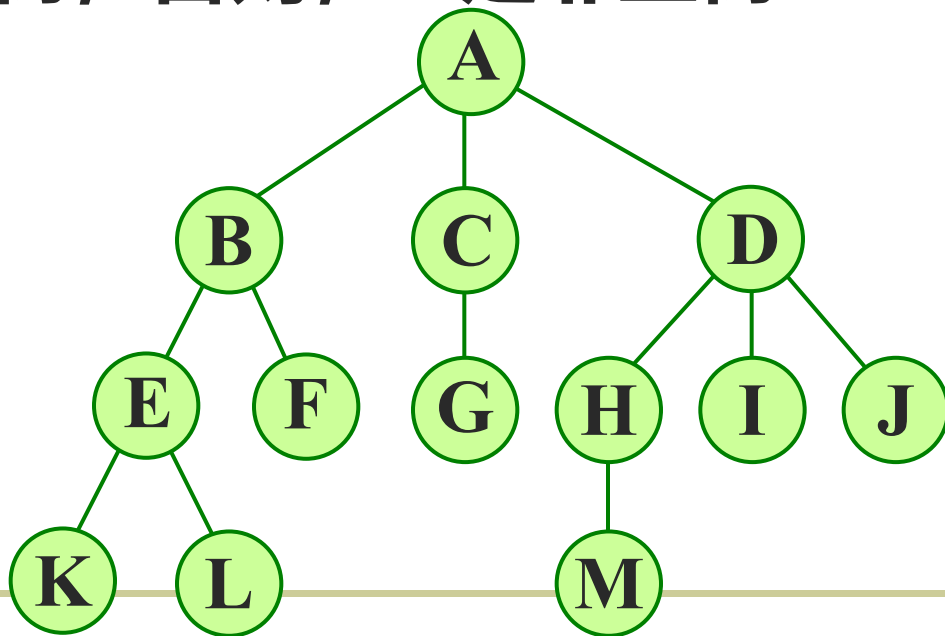
- 有根树:

一棵有根树  $T$ , 简称为树, 它是  $n$  ( $n \geq 0$ ) 个结点的有限集合。

当  $n = 0$  时,  $T$  称为空树; 否则,  $T$  是非空树

记作

$$T = \begin{cases} \phi, & n = 0 \\ \{r, T_1, T_2, \dots, T_m\}, & n > 0 \end{cases}$$





$$T = \begin{cases} \phi, & n = 0 \\ \{r, T_1, T_2, \dots, T_m\}, & n > 0 \end{cases}$$

- ◆  $r$  是一个特定的称为根(root)的结点, 它只有直接后继, 但没有直接前驱;
- ◆ 根以外的其他结点划分为  $m$  ( $m \geq 0$ ) 个互不相交的有限集合  $T_1, T_2, \dots, T_m$ , 每个集合又是一棵树, 并且称之为根的子树。
- ◆ 每棵子树的根结点有且仅有一个直接前驱, 但可以有0个或多个直接后继。



# 树的基本术语



- **子女**：若结点的子树非空，结点子树的根即为该结点的子女。
- **双亲（父亲）**：若结点有子女，该结点是子女的双亲（父亲）。
- **兄弟**：同一结点的子女互称为兄弟。
- **度**：
  - 结点的子女个数即为该**结点的度**；
  - 树中各个结点的度的最大值称为**树的度**。

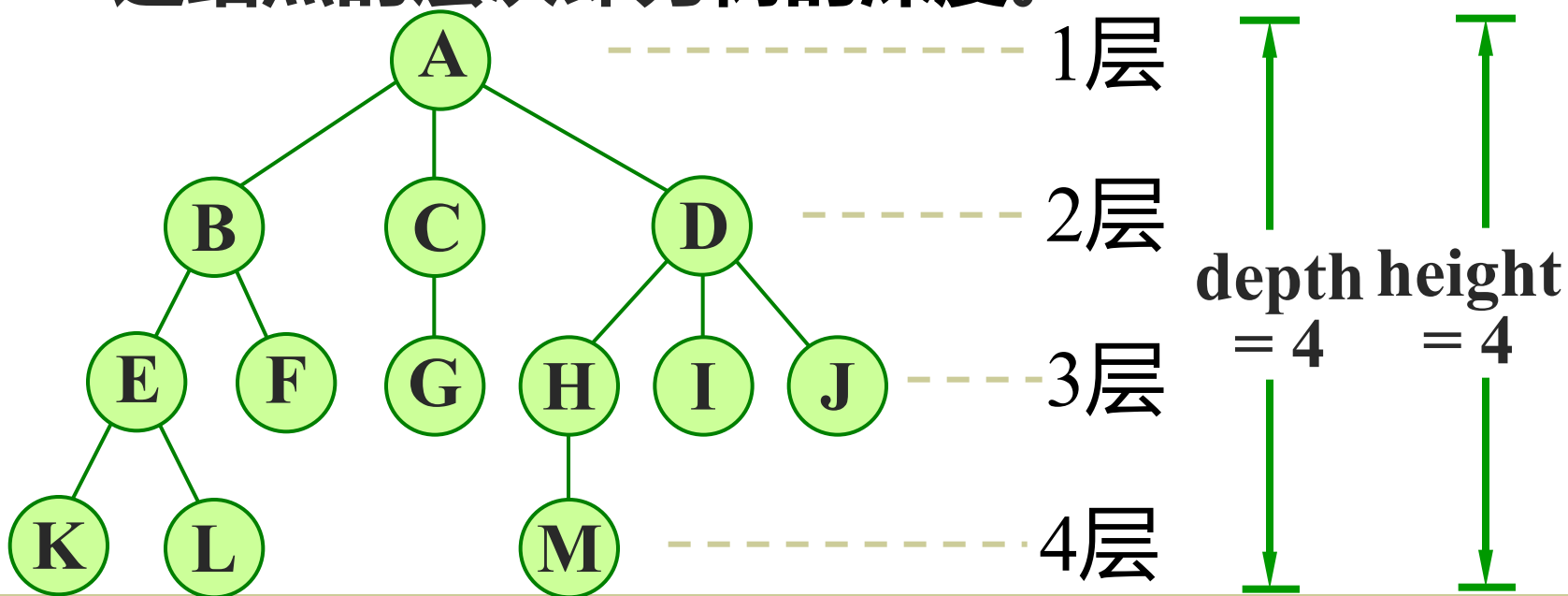




- **分支结点**：度不为0的结点即为分支结点，亦称为非终端结点。
- **叶结点**：度为0的结点即为叶结点，亦称为终端结点。
- **祖先**：根结点到该结点的路径上的各个结点都是该结点的祖先。
- **子孙**：某结点的所有下属结点，都是该结点的子孙。



- **结点的层次**：规定根结点在第一层，其子女结点的层次等于它的层次加一。以下类推。
- **深度**：结点的深度即为结点的层次；离根最远结点的层次即为树的深度。

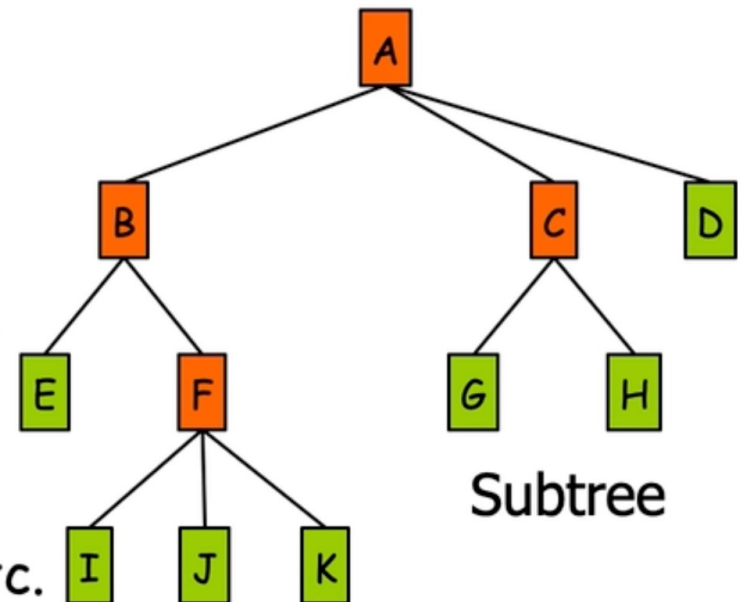




- **高度**：规定叶结点的高度为1，其双亲结点的高度等于它的高度加一。
- **树的高度**：等于根结点的高度，即根结点所有子女高度的最大值加一。
- **有序树**：树中结点的各棵子树  $T_0, T_1, \dots$  是有次序的，即为有序树。
- **无序树**：树中结点的各棵子树之间的次序是不重要的，可以互相交换位置。
- **森林**：森林是  $m$  ( $m \geq 0$ ) 棵树的集合。

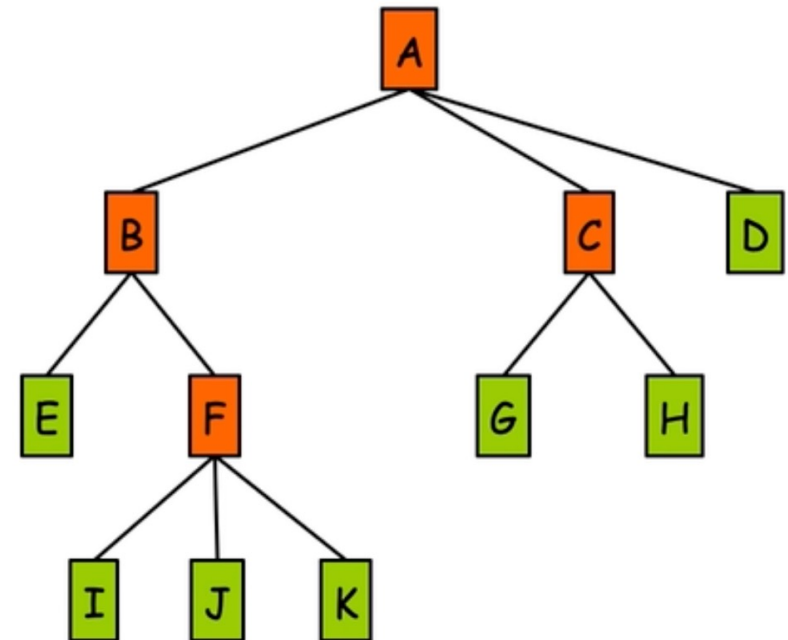


- ? Root:** node without parent (A)
- ? Siblings:** nodes share the same parent
- ? Internal node:** node with at least one child (A, B, C, F)
- ? External node (leaf):** node without children (E, I, J, K, G, H, D)
- ? Ancestors** of a node: parent, grandparent, grand-grandparent, etc.
- ? Descendant** of a node: child, grandchild, grand-grandchild, etc.





- ?** **Depth** of a node: number of ancestors
- ?** **Height** of a tree: maximum depth of any node
- ?** **Degree** of a node: the number of its children
- ?** **Degree** of a tree: the maximum number of its node.
- ?** **Subtree**: tree consisting of a node and its descendants





# Tree ADT

- ❑ We use positions to abstract nodes
- ❑ Query methods:
  - ❑ boolean **isInternal**(p)
  - ❑ boolean **isExternal**(p)
  - ❑ boolean **isRoot**(p)
- ❑ Generic methods:
  - ❑ integer **size**()
  - ❑ boolean **isEmpty**()
  - ❑ objectIterator **elements**()
  - ❑ positionIterator **positions**()
- ❑ Accessor methods:
  - ❑ position **root**()
  - ❑ position **parent**(p)
  - ❑ positionIterator **children**(p)
- ❑ Update methods:
  - ❑ **swapElements**(p, q)
  - ❑ object **replaceElement**(p, o)
- ❑ Additional update methods may be defined by data structures implementing the Tree ADT



# 树的抽象数据类型



```
template <class T>
```

```
class Tree {
```

```
//对象: 树是由 $n$  ( $\geq 0$ ) 个结点组成的有限集合。在
```

```
//类界面中的 position 是树中结点的地址。在顺序
```

```
//存储方式下是下标型, 在链表存储方式下是指针型。
```

```
//T 是树结点中存放数据的类型, 要求所有结点的
```

```
//数据类型都是一致的。
```

```
public:
```

```
    Tree ();
```

```
    ~Tree ();
```



```
BuildRoot (const T& value);  
    //建立树的根结点  
position FirstChild(position p);  
    //返回 p 第一个子女地址, 无子女返回 0  
position NextSibling(position p);  
    //返回 p 下一兄弟地址, 若无下一兄弟返回 0  
position Parent(position p);  
    //返回 p 双亲结点地址, 若 p 为根返回 0  
T getData(position p);  
    //返回结点 p 中存放的值  
bool InsertChild(position p, T& value);  
    //在结点 p 下插入值为 value 的新子女, 若插入  
    //失败, 函数返回false, 否则返回true
```





```
bool DeleteChild (position p, int i);  
    //删除结点 p 的第 i 个子女及其全部子孙结点,  
    //若删除失败, 则返回false, 否则返回true  
void DeleteSubTree (position t);  
    //删除以 t 为根结点的子树  
bool IsEmpty ();  
    //判树空否, 若空则返回true, 否则返回false  
void Traversal (void (*visit)(position p));  
    //遍历以 p 为根的子树  
};
```

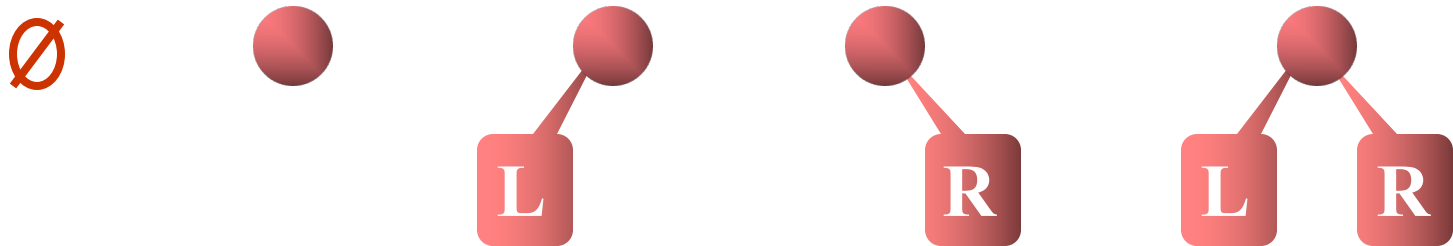


## 5.2 二叉树 (Binary Tree)



- 二叉树的定义

一棵二叉树是结点的一个有限集合，该集合或者为空，或者是由一个根结点加上两棵分别称为左子树和右子树的、互不相交的**二叉树**组成。



二叉树的五种不同形态



# 二叉树的性质



- **性质1** 若二叉树结点的层次从 1 开始, 则在二叉树的第  $i$  层最多有  $2^{i-1}$  个结点。 ( $i \geq 1$ )

[证明用数学归纳法]

- **性质2** 深度为  $k$  的二叉树最少有  $k$  个结点, 最多有  $2^k - 1$  个结点。 ( $k \geq 1$ )

因为每一层最少要有 1 个结点, 因此, 最少结点数为  $k$ 。

最多结点个数借助性质1: 用求等比级数前  $k$  项和的公式

$$2^0 + 2^1 + 2^2 + \dots + 2^{k-1} = 2^k - 1$$



- **性质3** 对任何一棵二叉树，如果其叶结点有  $n_0$  个，度为 2 的非叶结点有  $n_2$  个，则有

$$n_0 = n_2 + 1$$

若设度为 1 的结点有  $n_1$  个，总结点数为  $n$ ，总边数为  $e$ ，则根据二叉树的定义，

$$n = n_0 + n_1 + n_2 \quad e = 2n_2 + n_1 = n - 1$$

因此，有  $2n_2 + n_1 = n_0 + n_1 + n_2 - 1$

$$n_2 = n_0 - 1 \rightarrow$$

$$n_0 = n_2 + 1$$

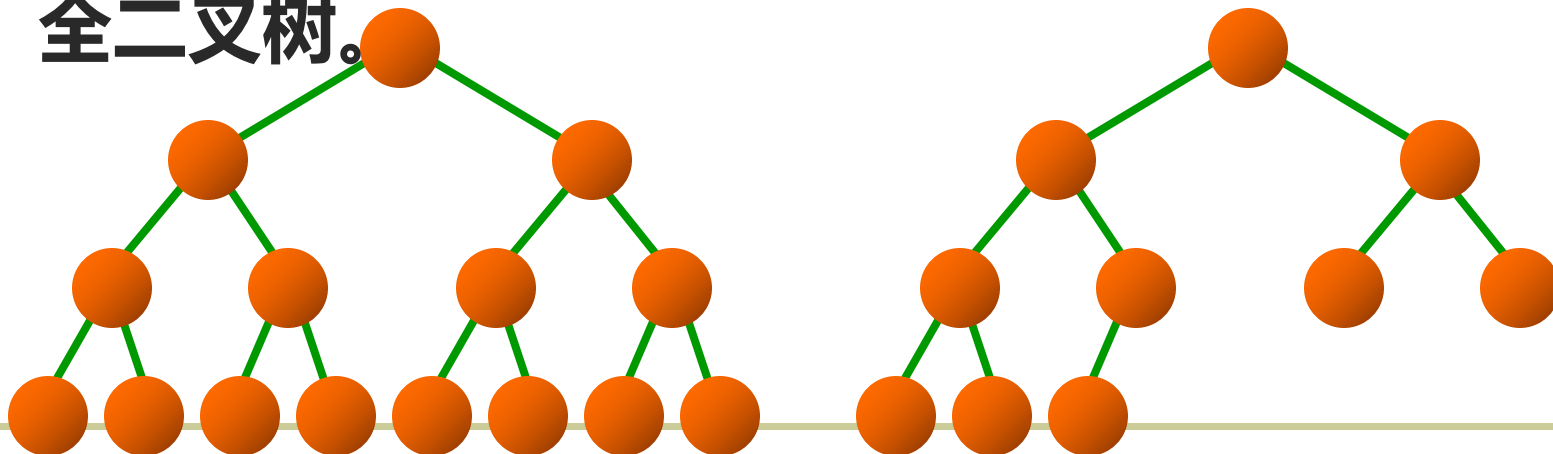


- **定义1 满二叉树 (Full Binary Tree)**

- 深度为  $k$  的满二叉树是有  $2^k - 1$  个结点的二叉树。

- **定义2 完全二叉树 (Complete Binary Tree)**

- 若设二叉树的深度为  $k$ ，则共有  $k$  层。除第  $k$  层外，其它各层 ( $1 \sim k-1$ ) 的结点数都达到最大个数，第  $k$  层从右向左连续缺若干结点，这就是完全二叉树。





**性质4** 具有  $n$  ( $n \geq 0$ ) 个结点的完全二叉树的深度为  $\lceil \log_2(n+1) \rceil$

设完全二叉树的深度为  $k$ , 则有

$$\underbrace{2^{k-1}-1}_{\text{上面 } k-1 \text{ 层结点数}} < n \leq \underbrace{2^k-1}_{\text{包括第 } k \text{ 层的最大结点数}}$$

上面  $k-1$  层结点数    包括第  $k$  层的最大结点数

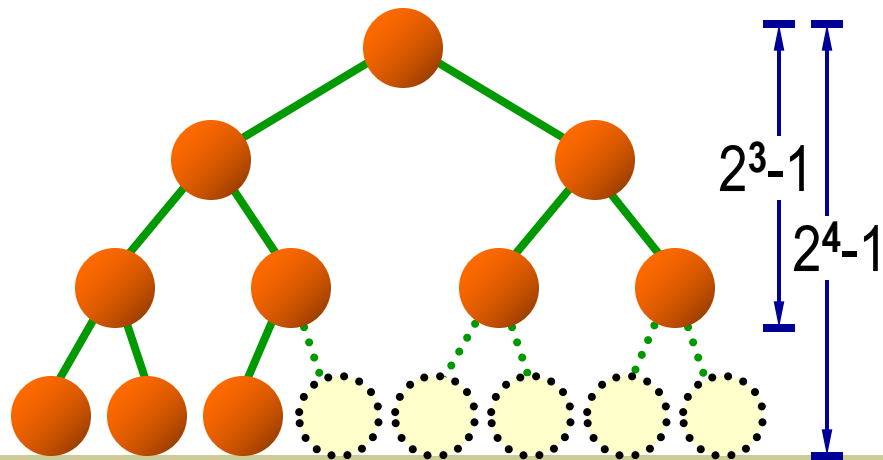
变形  $2^{k-1} < n+1 \leq 2^k$

取对数

$$k-1 < \log_2(n+1) \leq k$$

有

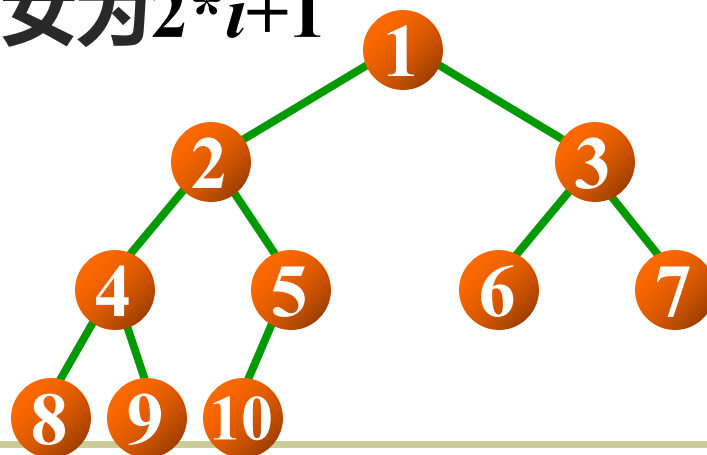
$$\lceil \log_2(n+1) \rceil = k$$





- **性质5** 如将一棵有 $n$ 个结点的完全二叉树自顶向下, 同一层自左向右连续给结点编号 $1, 2, \dots, n$ , 则任意编号 $i$ 有以下关系:

- ✓ 若 $i = 1$ , 则  $i$  无双亲
- ✓ 若 $i > 1$ , 则  $i$  的双亲为 $\lfloor i / 2 \rfloor$
- ✓ 若 $2*i \leq n$ , 则  $i$  的左子女为  $2*i$ ,  
若 $2*i+1 \leq n$ , 则  $i$  的右子女为 $2*i+1$
- ✓ 若  $i$  为奇数, 且 $i \neq 1$ ,  
则其左兄弟为 $i-1$
- ✓ 若  $i$  为偶数, 且 $i \neq n$ ,  
则其右兄弟为 $i+1$





- 在一棵度为4的树T中，若有20个度为4的结点，10个度为3的结点，1个度为2的结点，10个度为1的结点，则树T的叶结点个数是( )。
- 若一棵完全二叉树有768个结点，则该二叉树中叶结点的个数是( )。





# 二叉树的抽象数据类型



```
template <class T>
class BinaryTree {
//对象: 结点的有限集合, 二叉树是有序树
public:
    BinaryTree ();                //构造函数
    BinaryTree (BinTreeNode<T> *lch,
                BinTreeNode<T> *rch, T item);
    //构造函数, 以item为根, lch和rch为左、右子树
    //构造一棵二叉树
    int Height ();                //求树深度或高度
    int Size ();                  //求树中结点个数
```



|                                                 |               |
|-------------------------------------------------|---------------|
| <b>bool</b> IsEmpty ();                         | //判二叉树空否?     |
| BinTreeNode<T> *Parent (BinTreeNode<T> *t);     | //求结点 t 的双亲   |
| BinTreeNode<T> *LeftChild (BinTreeNode<T> *t);  | //求结点 t 的左子女  |
| BinTreeNode<T> *RightChild (BinTreeNode<T> *t); | //求结点 t 的右子女  |
| <b>bool</b> Insert (T item);                    | //在树中插入新元素    |
| <b>bool</b> Remove (T item);                    | //在树中删除元素     |
| <b>bool</b> Find (T& item);                     | //判断item是否在树中 |
| <b>bool</b> getData (T& item);                  | //取得结点数据      |



```
BinTreeNode<T> *getRoot (); //取根
void preOrder (void (*visit) (BinTreeNode<T> *t));
    //前序遍历, visit是访问函数
void inOrder (void (*visit) (BinTreeNode<T> *t));
    //中序遍历, visit是访问函数
void postOrder (void (*visit) (BinTreeNode<T> *t));
    //后序遍历, visit是访问函数
void levelOrder (void (*visit)(BinTreeNode<T> *t));
    //层次序遍历, visit是访问函数
};
```